

Αναφορά Project hy335a 2023
Νικολέτα Ξενάκη csd4968, Βικτωρία Μηλιαράκη csd4944

Εισαγωγικά

Στο φετινό πρότζεκτ υλοποιήσαμε το πρωτόκολλο TCP στην γλώσσα C και χρησιμοποιώντας ως βάση τις συναρτήσεις του UDP (sendto() , recvfrom()). Ως compiler χρησιμοποιήθηκε ο gcc χωρίς περαιτέρω flags. Το port που δόθηκε ήταν το 3636. Γίνεται compile με το Makefile στον φάκελο test (make all). Και μετά λόγω των arguments τρέχει με το χερί.

Υλοποίηση

Πιο συγκεκριμένα υλοποιήσαμε τον μηχανισμό flow control που ελέγχει τον ρυθμό μετάδοσης των δεδομένων ώστε να συγχρονίζεται αποστολέας και παραλήπτης και να μην ξεχειλίζει ο receive buffer του παραλήπτη, υλοποιήσαμε τον μηχανισμό για congestion control με τον οποίο ο αποστολέας ελέγχει τον ρυθμό αποστολής πακέτων σε περίπτωση συμφόρησης δικτύου και τέλος τις τρεις πεπερασμένες καταστάσεις στις οποίες εισέρχεται το πρωτόκολλο **TCP** για να διασφαλίσει την σωστή και έγκαιρη παράδοση των πακέτων στον παραλήπτη. Ειδικότερα ο client μόνο μπορεί να στέλνει πακέτα στον server με μορφή pipelining και έπειτα περιμένει τα κατάλληλα acknowledgments απ' αυτόν. Αρχικά ο client στέλνει ένα πακέτο με SYN=1 , ο server το λαμβάνει, ελέγχει αν το πακέτο είναι corrupted και ανταποκρίνεται και αυτός με ένα πακέτο ACK προς τον client όπου και αυτό έχει flag SYN=1. Ο client το λαμβάνει και έτσι η σύνδεση μπαίνει σε κατάσταση ESTABLISHED. Τώρα μπορεί ο client να στέλνει πακέτα με δεδομένα στον server και αυτός να τα λαμβάνει στέλνοντας τα κατάλληλα acknowledgments. Σε περίπτωση timeout και duplicate ACKs από τον παραλήπτη το TCP εισέρχεται στις αντίστοιχες καταστάσεις και στέλνει ξανά το τελευταίο πακέτο που χάθηκε. Καθώς ο παραλήπτης λαμβάνει πακέτα κάποια στιγμή θα του έρθει ένα με flag FIN=1 που σημαίνει ότι ο αποστολέας θέλει να τερματίσει την σύνδεση. Τότε, αν το πακέτο αυτό ληφθεί από τη microtcp_recv(), θα επιστρέψει -1 και θα θέσει το state του socket ως CLOSING_BY_PEER. Έπειτα η microtcp_shutdown() είναι υπεύθυνη για τη συνέχεια του τερματισμού της σύνδεσης: μόλις ο αποστολέας λάβει το ACK από τον παραλήπτη, θέτει το state του socket ως CLOSING_BY_HOST. (Αν υπάρξει κάποιο πρόβλημα με τη λήψη των πακέτων το state γίνεται INVALID.) Μετά περιμένει να λάβει ένα πακέτο FIN=1 από τον παραλήπτη και μόλις το λάβει του απαντάει με ένα πακέτο ACK και θέτει το state της σύνδεσής του σε CLOSED. Τέλος, όταν αντίστοιχα λάβει ο παραλήπτης το ACK, θέτει το state της σύνδεσής του και αυτός σε CLOSED.

Παρατηρήσεις

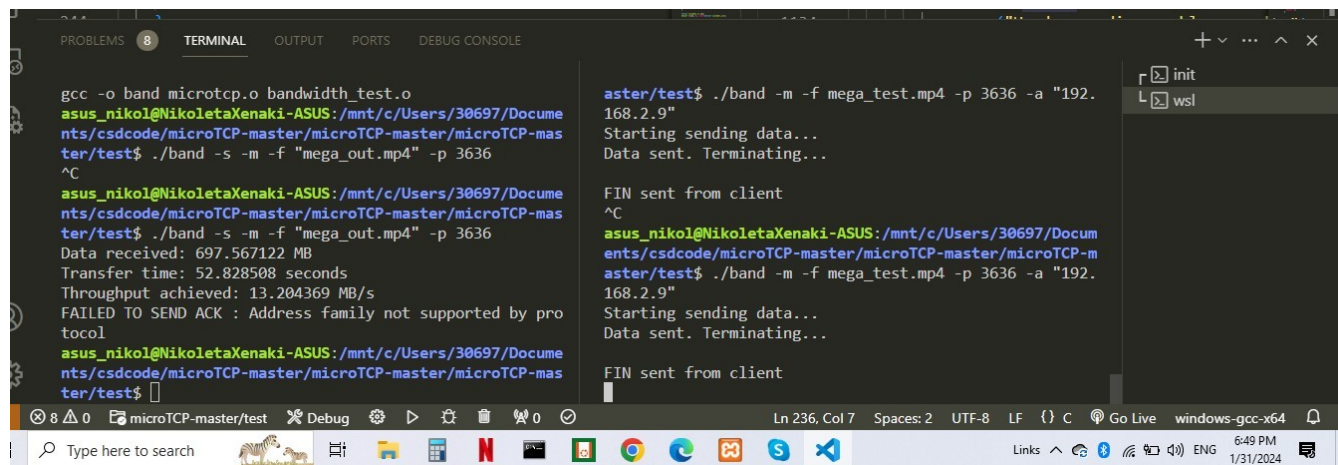
Κατά την εκτέλεση του μεγαλύτερου τεστ (700MB περίπου), locally, το πρόγραμμα έτρεξε κανονικά μεταφέροντας όλα τα δεδομένα. Ωστόσο παρατηρήθηκαν προβλήματα κατά τη διάρκεια τερματισμού στη `microtcp_shutdown()`. Στη περίπτωση που το FIN έχει ληφθεί από την `receive`, όταν ο server στέλνει το επόμενο πακέτο (ACK), η destination address ,μόνο, γίνεται corrupted και το πακέτο χάνεται με αποτέλεσμα ο client να μη το λαμβάνει. Παρόλα αυτά έχουμε δοκιμάσει την ίδια συνάρτηση σε απλούστερη περίπτωση, όπως στη 1η φάση, και λειτουργεί κανονικά. Τέλος παρόλλο που χρησιμοποιήσαμε Ubuntu -22.04 terminal εμείς, στο μηχανήμα styx όταν ο server έκανε `receive` το πρώτο header πετούσε segmentation fault. Από μηνύματα debugging καταλάβαμε ότι ευθυνόνταν η γραμμή:

```
header=((microtcp_header_t*)(receive_buf));
```

Αν και ο `receive_buf` δεσμεύεται κανονικά και με το σωστό μέγεθος.

Performance=micro_tcp/tcp = 13.204369/384.340072(MB/s)=0.35(MB/s) απόκλιση.

Αποδεικτικά ότι όντως μας έτρεχε εκτός την shutdown:

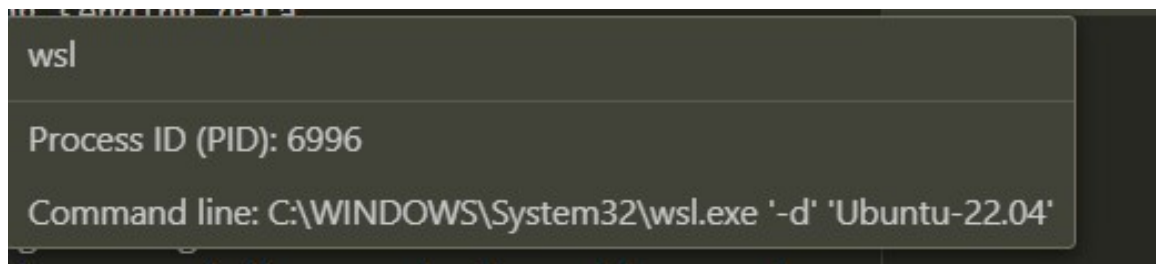


```
gcc -o band microtcp.o bandwidth_test.o
asus_nikol@NikoletaXenaki-ASUS:/mnt/c/Users/30697/Documents/csdcode/microTCP-master/microTCP-master/microTCP-master/test$ ./band -s -m -f "mega_out.mp4" -p 3636
^C
asus_nikol@NikoletaXenaki-ASUS:/mnt/c/Users/30697/Documents/csdcode/microTCP-master/microTCP-master/microTCP-master/test$ ./band -s -m -f "mega_out.mp4" -p 3636
Data received: 697.567122 MB
Transfer time: 52.828508 seconds
Throughput achieved: 13.204369 MB/s
FAILED TO SEND ACK : Address family not supported by protocol
asus_nikol@NikoletaXenaki-ASUS:/mnt/c/Users/30697/Documents/csdcode/microTCP-master/microTCP-master/microTCP-master/test$

aster/test$ ./band -m -f mega_test.mp4 -p 3636 -a "192.168.2.9"
Starting sending data...
Data sent. Terminating...

FIN sent from client
^C
asus_nikol@NikoletaXenaki-ASUS:/mnt/c/Users/30697/Documents/csdcode/microTCP-master/microTCP-master/microTCP-master/test$ ./band -m -f mega_test.mp4 -p 3636 -a "192.168.2.9"
Starting sending data...
Data sent. Terminating...

FIN sent from client
```

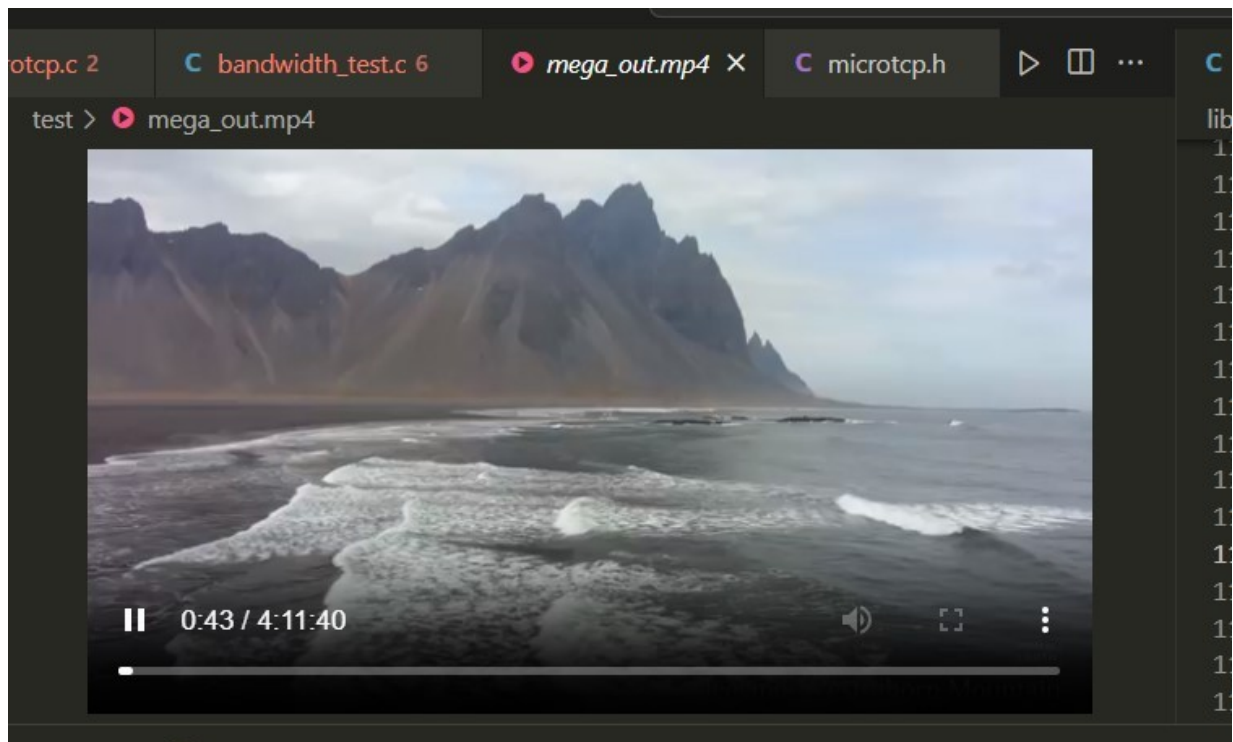


```
wsl

Process ID (PID): 6996

Command line: C:\WINDOWS\System32\wsl.exe '-d' 'Ubuntu-22.04'
```

To output:



ΤΕΛΟΣ!

ΕΥΧΑΡΙΣΤΟΥΜΕ!